

Daily Progress Report

Week 5_Day4: Routing & Navigation

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_5 (Day 4)
Date	24 July, 2026

Day 4: Routing & Navigation

1.1 Topics Studied

- Understanding client-side routing in React using react-router-dom
- Setting up BrowserRouter as the root routing provider
- Defining application routes using Routes and Route
- Creating navigation between pages using Link and NavLink
- Programmatic navigation using the useNavigate hook
- Implementing a responsive navigation bar with active route highlighting
- Configuring dynamic routing for undefined URLs using a 404 Not Found page
- Organizing React applications into reusable components and page-based architecture
- Managing page navigation without full-page reloads for improved performance
- Applying responsive layouts and modern UI design principles to a multi-page React application

1.2 Task Description

Developed a professional multi-page React application demonstrating client-side routing using **react-router-dom**. The application includes dedicated **Home** and **About** pages connected through a responsive navigation bar. Navigation is implemented using both declarative links and programmatic routing with the **useNavigate** hook. Additionally, a dynamic **404 Not Found** page handles invalid URLs gracefully by informing users when a requested page does not exist and providing an option to return to the homepage. The project follows a modular folder structure with separate components, pages, and styling files to reflect real-world React development practices.

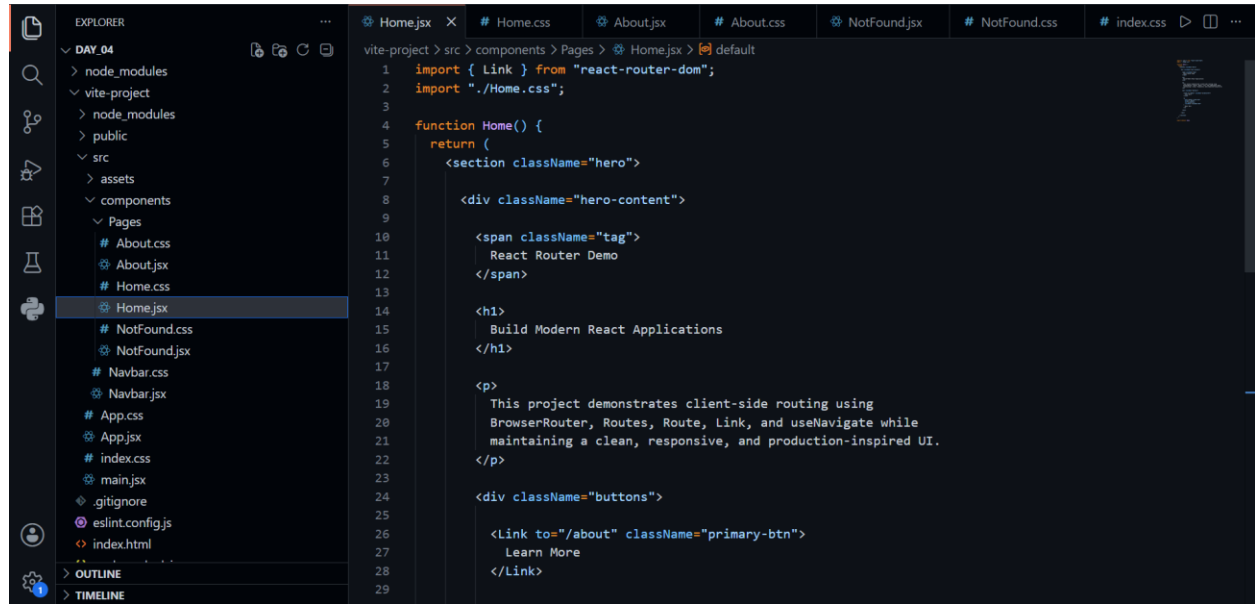
1.3 Implementation Details

- Configured **BrowserRouter** in the application's entry point to enable client-side routing.
- Defined application routes using the **Routes** and **Route** components for the Home, About, and 404 Not Found pages.
- Developed a reusable **Navbar** component containing navigation links for seamless page transitions.
- Implemented **NavLink** to automatically highlight the currently active navigation item.
- Created a modern Home page featuring a professional hero section with descriptive content and navigation buttons.
- Developed an About page explaining the application's purpose and demonstrating programmatic navigation using the **useNavigate** hook.
- Configured a wildcard (*) route to display a custom 404 Not Found page whenever users attempt to access an undefined URL.
- Implemented a "Return Home" button on the 404 page using **useNavigate** for programmatic redirection.
- Organized the application into reusable folders including components, pages, and dedicated CSS files to improve maintainability and scalability.
- Designed a fully responsive user interface with custom CSS featuring a sticky navigation bar, modern card layouts, hover animations, active navigation states, and mobile-friendly spacing.

1.4 Outcome

Successfully developed a professional multi-page React application that demonstrates modern client-side routing using **react-router-dom**. The project reinforced the understanding of configuring routing with **BrowserRouter**, defining application routes, navigating using **Link**, **NavLink**, and **useNavigate**, and handling undefined routes through a custom **404 Not Found** page. Additionally, organizing the project into reusable components and implementing a responsive user interface strengthened knowledge of scalable React application architecture and real-world frontend development practices.

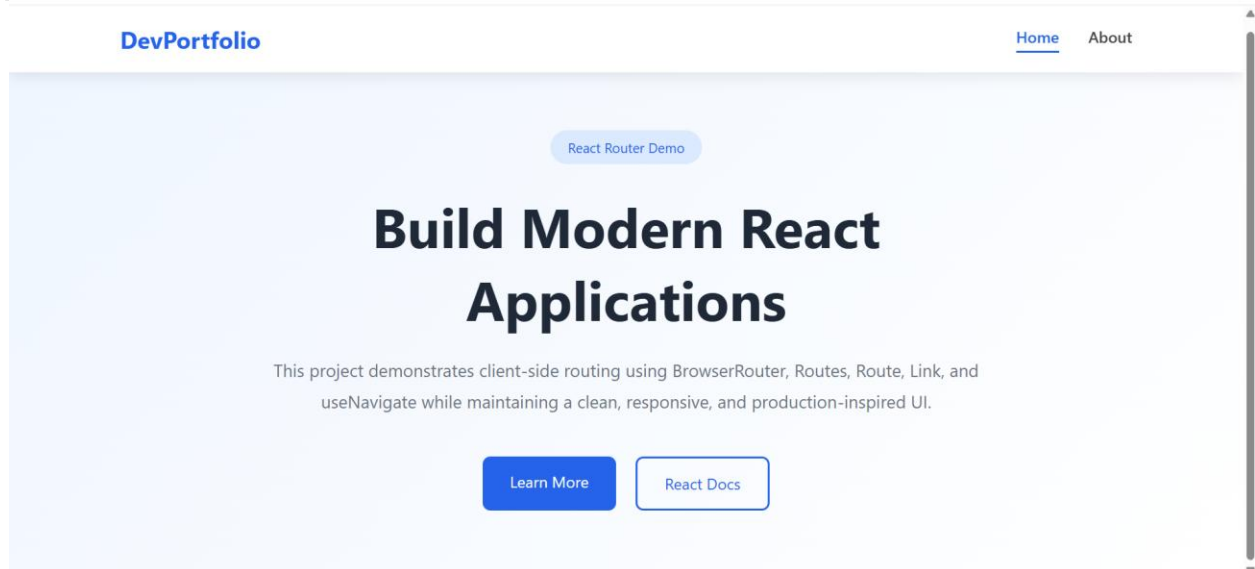
Screenshots:



The screenshot shows a VS Code editor with the Explorer sidebar on the left. The Explorer sidebar shows a file tree for a project named 'vite-project'. The file tree includes folders like 'node_modules', 'vite-project', 'public', 'src', 'assets', 'components', and 'Pages'. The 'Pages' folder is expanded, showing files like 'About.css', 'About.jsx', 'Home.css', 'Home.jsx', 'NotFound.css', 'NotFound.jsx', 'Navbar.css', 'Navbar.jsx', 'App.css', 'App.jsx', 'index.css', 'main.jsx', and '.gitignore'. The 'Home.jsx' file is selected. The main editor area shows the code for 'Home.jsx'. The code is as follows:

```
1 import { Link } from "react-router-dom";
2 import "../Home.css";
3
4 function Home() {
5   return (
6     <section className="hero">
7       <div className="hero-content">
8         <span className="tag">
9           React Router Demo
10         </span>
11         <h1>
12           Build Modern React Applications
13         </h1>
14         <p>
15           This project demonstrates client-side routing using
16           BrowserRouter, Routes, Route, Link, and useNavigate while
17           maintaining a clean, responsive, and production-inspired UI.
18         </p>
19         <div className="buttons">
20           <Link to="/about" className="primary-btn">
21             Learn More
22           </Link>
23         </div>
24       </div>
25     </section>
26   );
27 }
28
29 export default Home;
```

Screenshots:



About This Project

This application demonstrates modern client-side routing in React using react-router-dom. Instead of reloading the page, React updates only the necessary components, creating a fast and smooth user experience.

✓ BrowserRouter

✓ Routes & Route

✓ Link Navigation

✓ useNavigate Hook

✓ Dynamic 404 Page

← Back Home